

Disk Identity and Integrity

Pushkar Chitnis

(Azure Confidential Computing)

Scenario

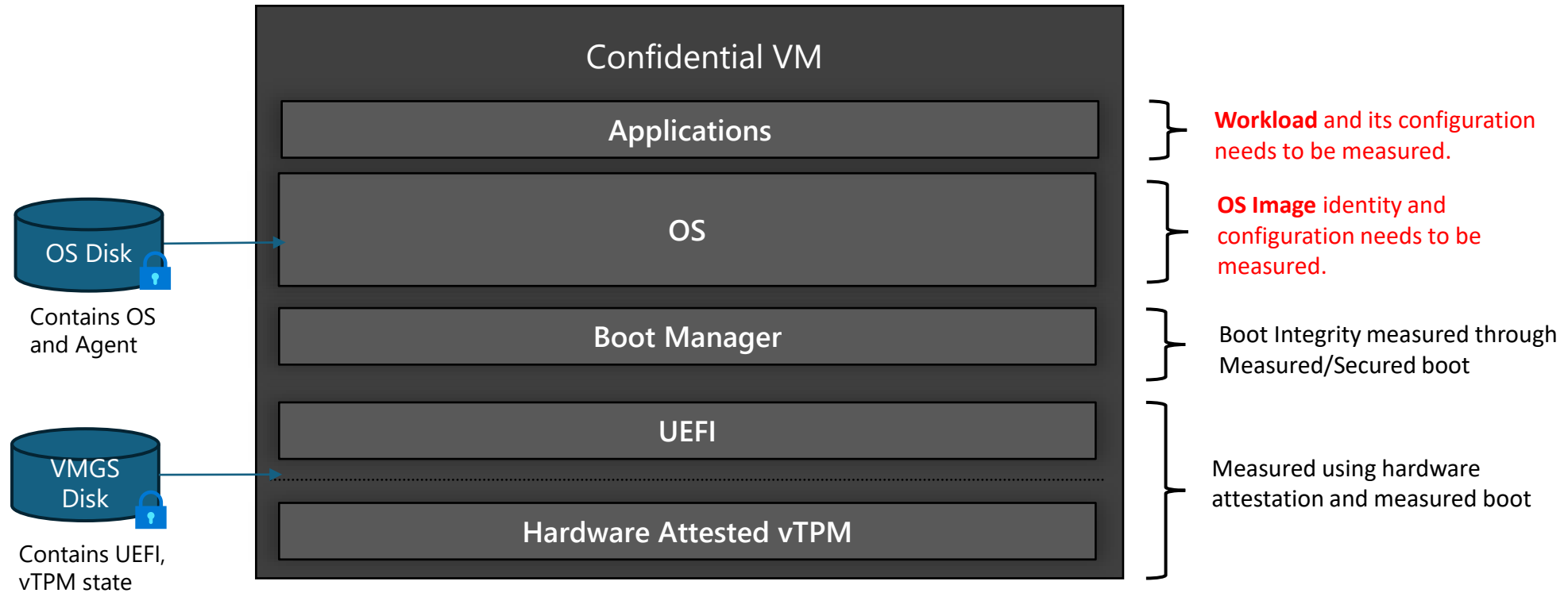
Disk state and configuration are part of effective Confidential Trust Boundary of a Confidential VM.

- To reason about security state of a Virtual Machine, we need to reason about security state of its disks.
 - Who published the OS image used to boot the VM?
 - Can we establish a cryptographic identity for an OS Disk?
 - Can we assert this identity, if disk's ReadWrite state is persisted across reboots?
 - Can we generalize above primitives to any disks attached to VM?

We are interested in identifying collaborators / partners interested in this problem space.

Confidential VM Stack

TCB Components and properties which need to be captured in attestation

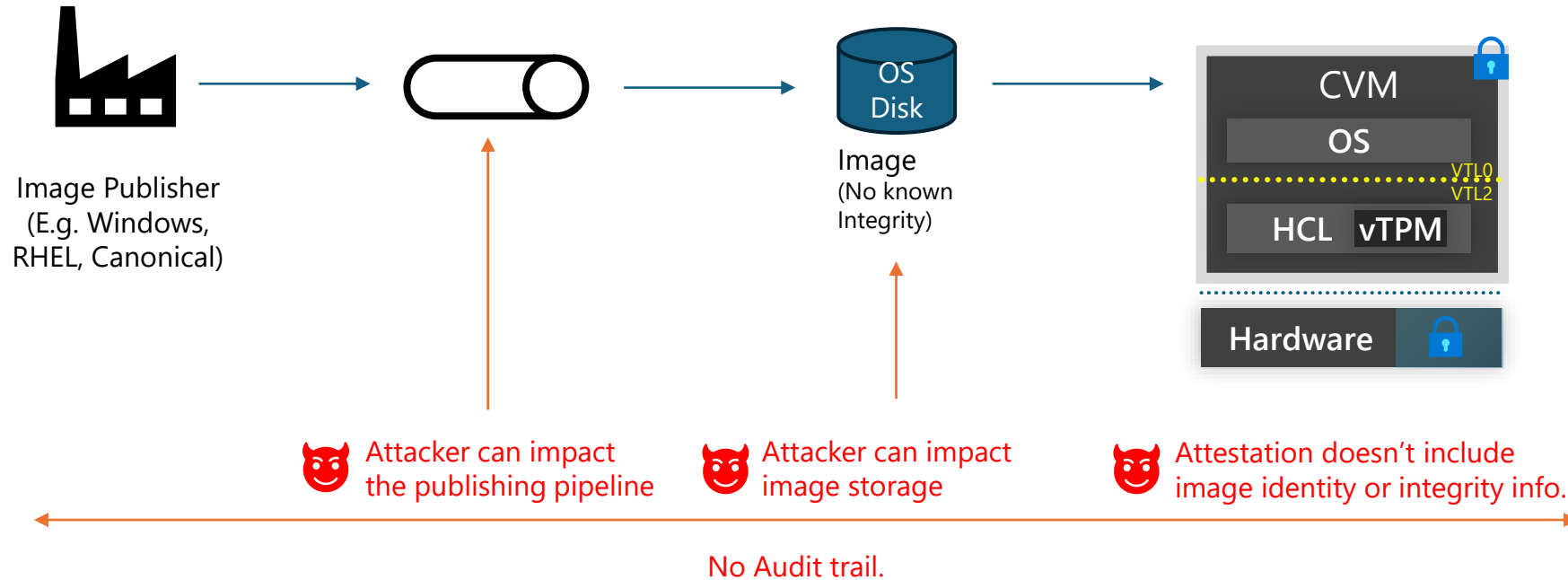


Azure vTPM attestation and binding:

https://github.com/CCC-Attestation/meetings/blob/main/materials/MikeStunes_vTPM_Attestation.pdf

OS Disk Integrity

OS disk integrity is critical to reason about confidential VM security posture

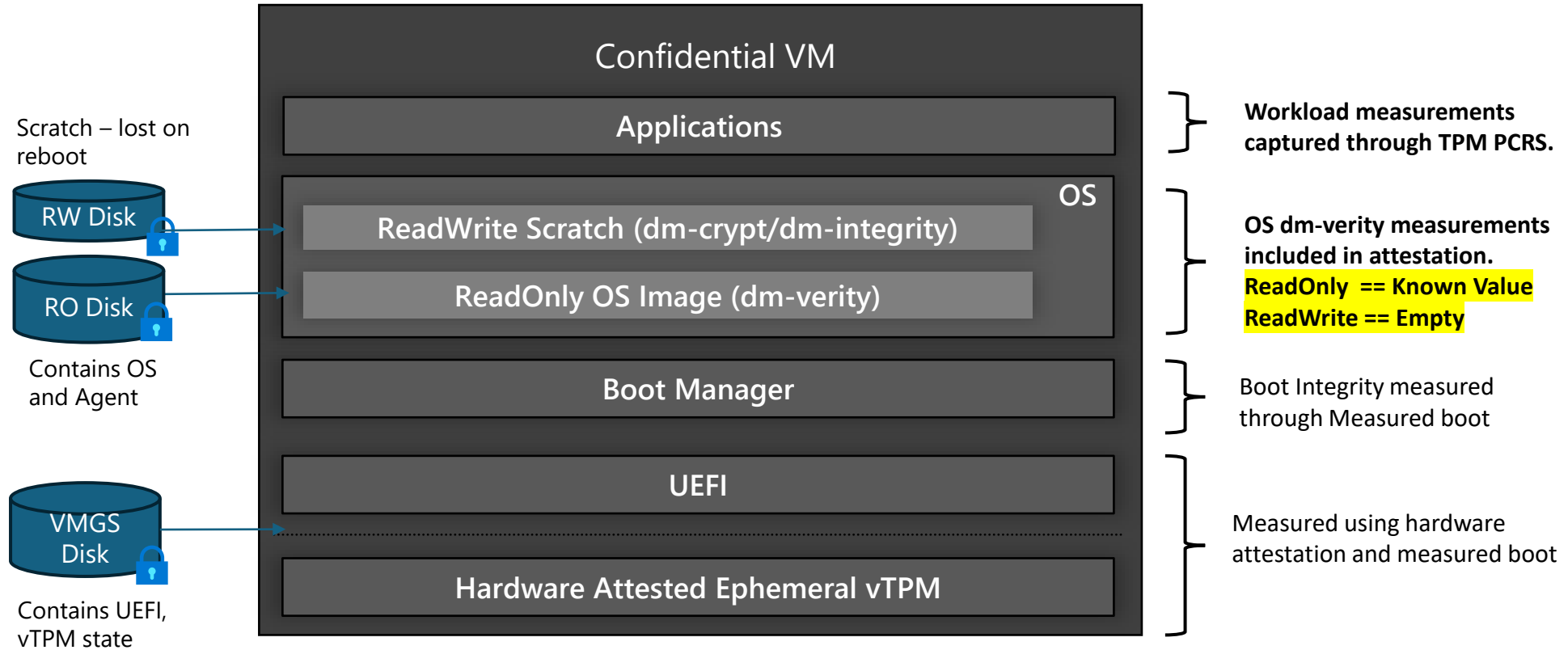


Integrity means that all state transitions of the OS disk are authorized

- The disk starts from a **known initial state**, and
- **Every subsequent modification** to that disk is **caused only by software executing inside the Conf. VM**.
- No external entity (host or control plane) can modify, replace, or roll back disk state **without detection**.

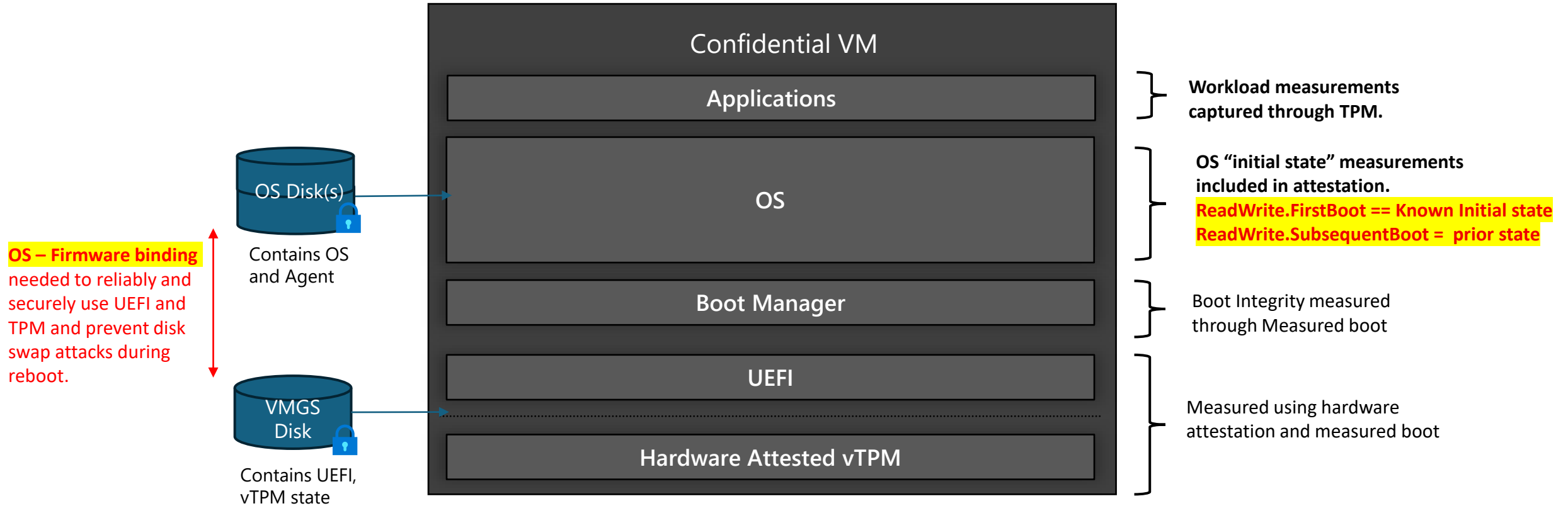
Confidential VM – Purely Ephemeral

In purely ephemeral configuration, all state is reset to known good integrity state across reboots.



Confidential VM – Persisted

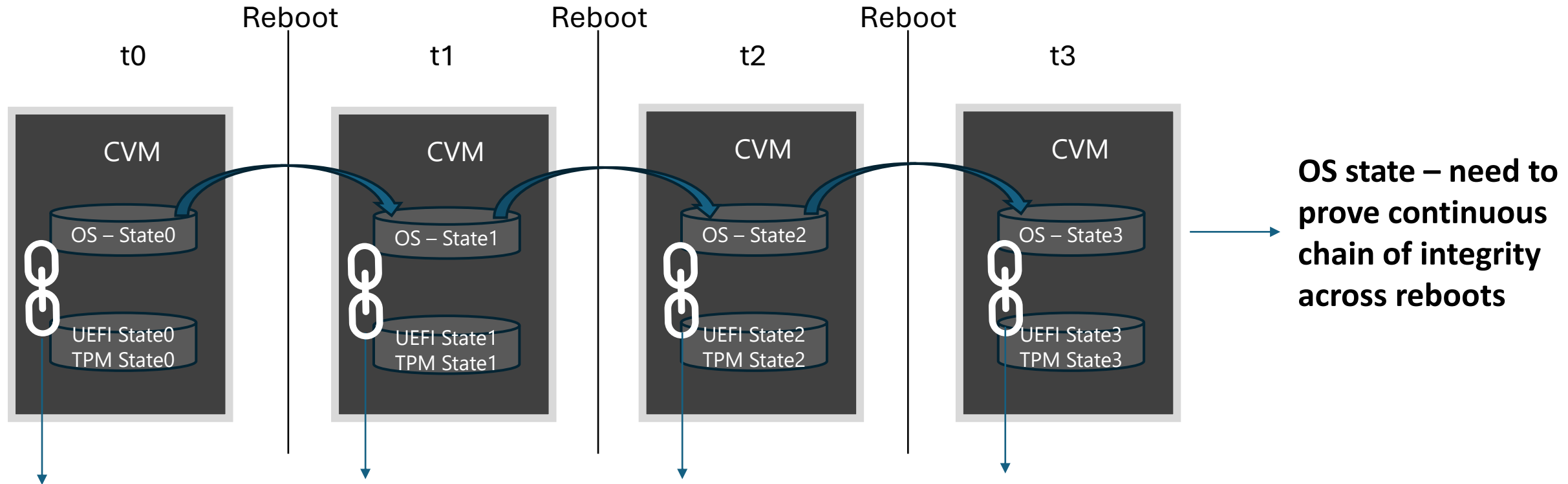
In persisted configuration, all state is retained across reboots.



Note: There is no re-initialization across reboots.

Confidential VM – Persisted state

In persisted configuration, all OS state is retained across reboots.



OS state – Firmware state need cryptographic binding to enable reliable and secure use of TPM and prevent state swap or state rollback attacks.

Any rollbacks or replays, if allowed, are for the entire OS state/firmware state collective.

OS Disk Identity and Integrity

Verify OS disk identity and Integrity and attest at boot and reboot

- OS Disk Identity
 - “Who published the image?”
 - Identity includes Initial OS Image hash, attributes and signer.
- Runtime time Integrity
 - Protect against external disk tampering during runtime.
 - Maintain continuous ordered integrity chain thru lifetime of CVM
- Attestation: Attest OS Disk Identity through boot and reboot.
 - Attestation should guarantee that OS disk was provisioned using specified Image.
- Bind: Ensure firmware/vTPM state and OS state are bound to each other.

Existing solutions for Disk Identity and Integrity

Disk Integrity is bound to “Initial disk Identity” and “Runtime Integrity”

- Existing solutions relying on layered architecture

Primitives	Description	Linux	Windows
RO Base Disk	Typically, RO Merkel implementation	dm-verity	CimFS
Layering	Create layering scheme with RO(s) and RW	overlay-fs or dm-snapshot	UnionFS
RW Disk Encryption	Encrypt writes using key bound to platform integrity and RO Disk measurements.	dm-crypt	Bitlocker
RW Disk Integrity	Integrity protect using HMAC bound to platform integrity and RO Disk measurements.	dm-integrity	--

- RW Integrity:
 - No solution for block device level integrity to prevent block swap or block rollback.
 - No scheme to create chain of integrity across edits/mounts.

Use RW Merkel for Disk Identity and Integrity

Disk Integrity is bound to “Initial disk Identity” and “Runtime Integrity”

- “RW Merkel” allows representation of a RW disk using a single root hash.
 - Merkel root hash covers both data and metadata.
 - Any update to data or metadata changes the root hash.
Note: RW mount changes metadata and thus changes root hash of the RW Merkel.
- RW Merkel implementations
 - Linux: ZFS (though not up streamed), BTRFS*
 - Windows: ReFS (Windows Server 2025+ offers sha256 mode).

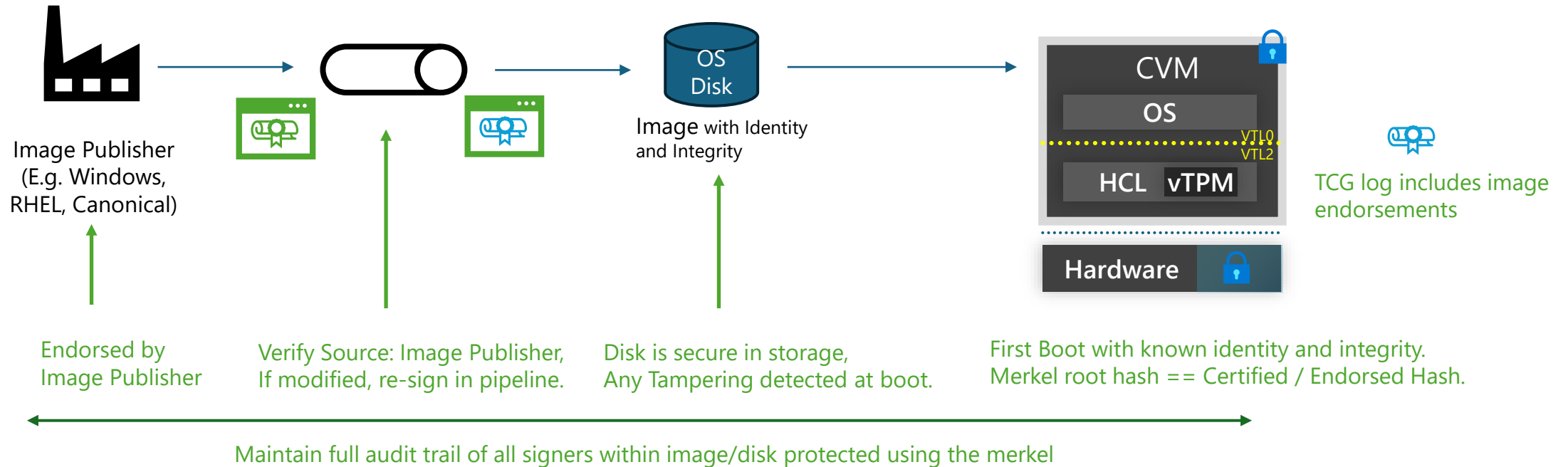
Use RW Merkel for Disk Identity and Integrity

Disk Integrity is bound to “Initial disk Identity” and “Runtime Integrity”

- Initial Disk Identity is combination of RO and RW.
 - [RO Merkel Hash] + [RW Merkel Hash] where RW Hash == [initialized to empty].
 - [RO Merkel Hash] + [RW Merkel Hash] where RW Hash == [trusted value/signature].
 - [RW Merkel Hash] where RW Hash == [trusted value/signature]
(No RO or layering)
- Runtime Integrity at FS level or Block Device level is covered using RW Merkel
 - A merkel root hash signed by trusted entity is trusted.
 - Signing and verifying root hash can be used to create a trusted integrity chain.

OS Publishing pipeline

OS publishing pipeline is enhanced with all updates represented using an endorsed merkel root hash.

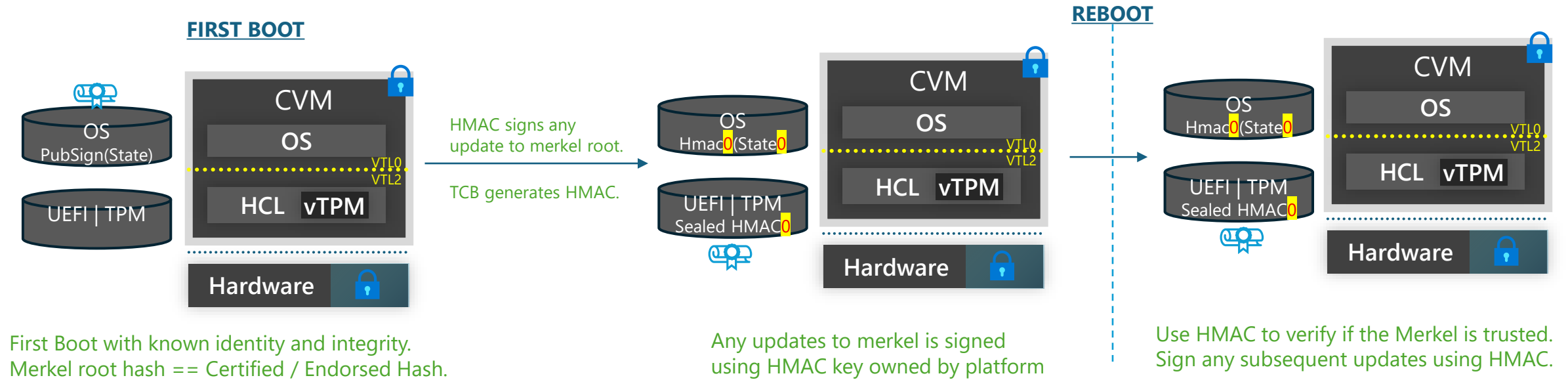


Identity and Integrity chain maintained within the publishing pipeline

- Publisher generates image with known root hash, Also generates endorsements for the root hash.
- Any entity in publishing pipeline can modify contents of image as long as it
 - Verifies source of disk it received, before modification.
 - Endorses the disk, after modification.
- The CVM at first boot is able to verify the merkel root hash matches latest endorsement.

OS Boot → Reboot integrity chain

OS boot and reboot can maintain an integrity chain by signing merkel root hash with HMAC owned by platform.



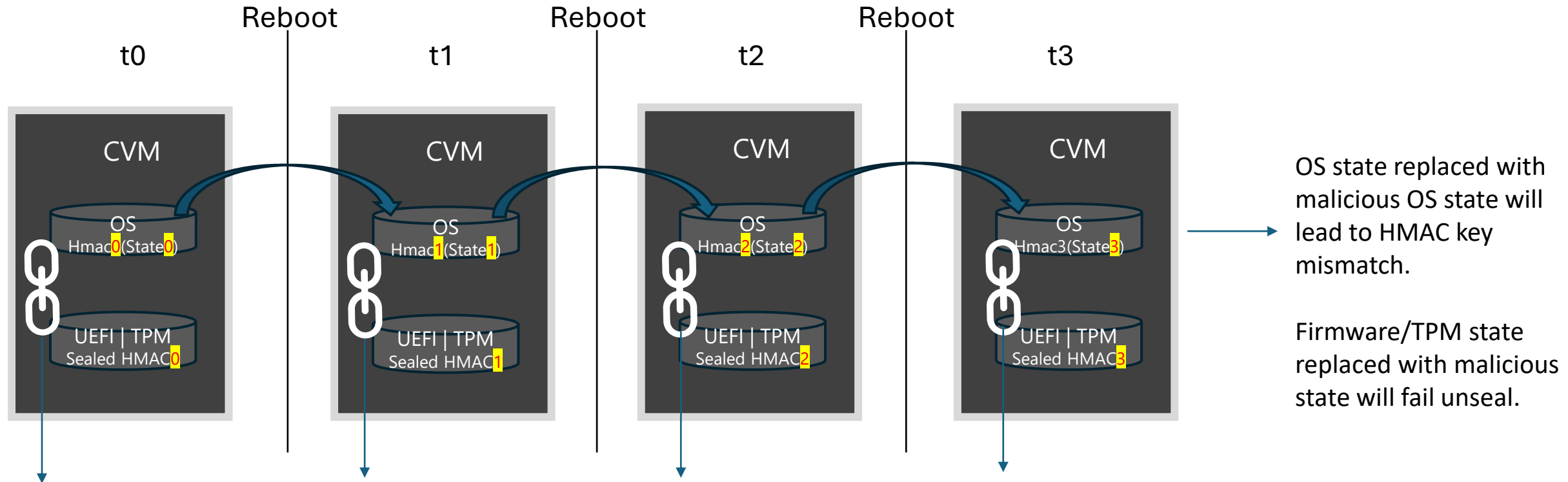
Identity and Integrity chain maintained by CVM's Trusted platform TCB using HMAC Key

- Every mount includes derivation of HMAC key and verification of root hash.
- Boot components can generate HMAC key as follows:
 - HMAC key can be bound to boot integrity (e.g. Secret Seed sealed to TPM PCRs)
 - HMAC key can be derived using Initial Image Identity as AuthData.

HMAC Key (for runtime integrity) = KDF(Seed)[Initial Image Measurements]
where Seed: is bound to TPM PCRs (representing boot integrity).

OS Disk Binding and rollback protection

OS and firmware state is cryptographically bound such that boot components can detect swaps



HMAC key can be rotated every boot to ensure that state across reboot is aligned and bound to new HMAC Key. Misaligned rollback of individual state will lead to HMAC key mismatch.

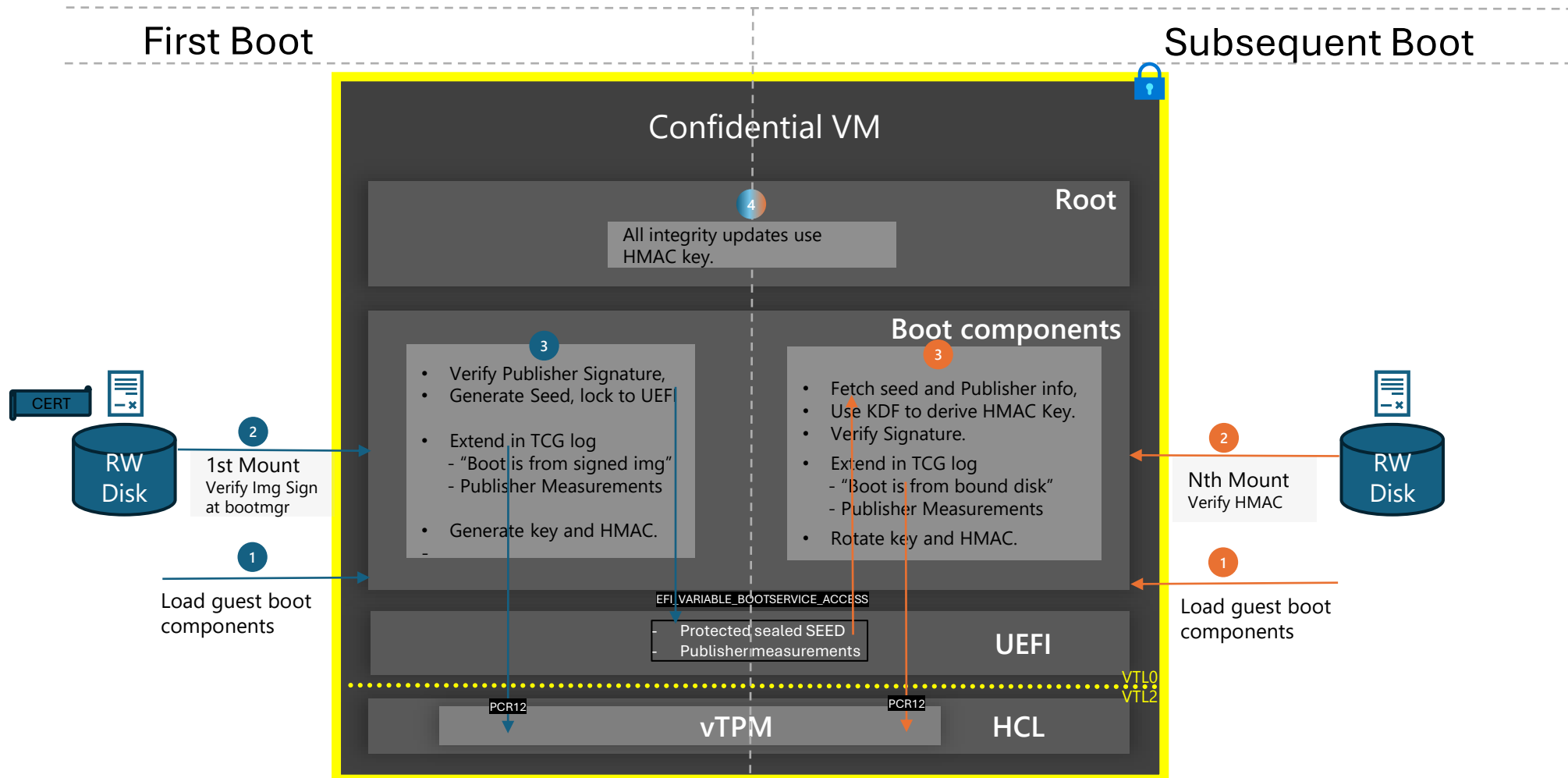
Disk Identity and Integrity

Disk Integrity is bound to “Initial disk Identity” and “Runtime Integrity”

- For Disk Identity and integrity, we rely on
 - **Initial Image Identity**: Authenticity of publisher (keys/signatures) of Image verified at first boot.
 - **Runtime integrity Continuity**: Authenticity of HMAC key for runtime Integrity and integrity chain post reboots.
- Authenticity needs to be verified and attested by trusted entity withing CVM.
 - For OS disk, we rely on trusted boot within CVM e.g. UEFI, UKI, Boot Manager.
 - For Data disk, we can rely on Boot integrity+ OS environment.
- HMAC key can be bound to single trusted system or collection of trusted systems.

OS Disk Identity and Integrity

OS Disk Integrity is bound to “Initial Image Identity” and “OS runtime integrity”



Interim Solutions without RW Merkel

Disk Integrity is bound to “Initial disk Identity” and “Runtime Integrity”

- Without RW Merkel, we can still use primitives such as
 - Establish Initial identity using RO Merkel and
 - Have runtime encryption/block level integrity bound to derived key.
- The model will NOT be able to protect against
 - Block Rollback / Block swap.
 - OS / Firmware VHD rollback (unless block level HMAC is updated on every mount)
- Machines boot Image with UKI + RO Merkel can choose to do the following:
 - At first Boot → Create Empty RW overlay using dm-snapshot (or overlay-fs)
 - At first Boot → Use dm-clone to create RW block device bound to hmac.

Conclusion

Disk state and configuration are part of effective Confidential Trust Boundary of a Confidential VM.

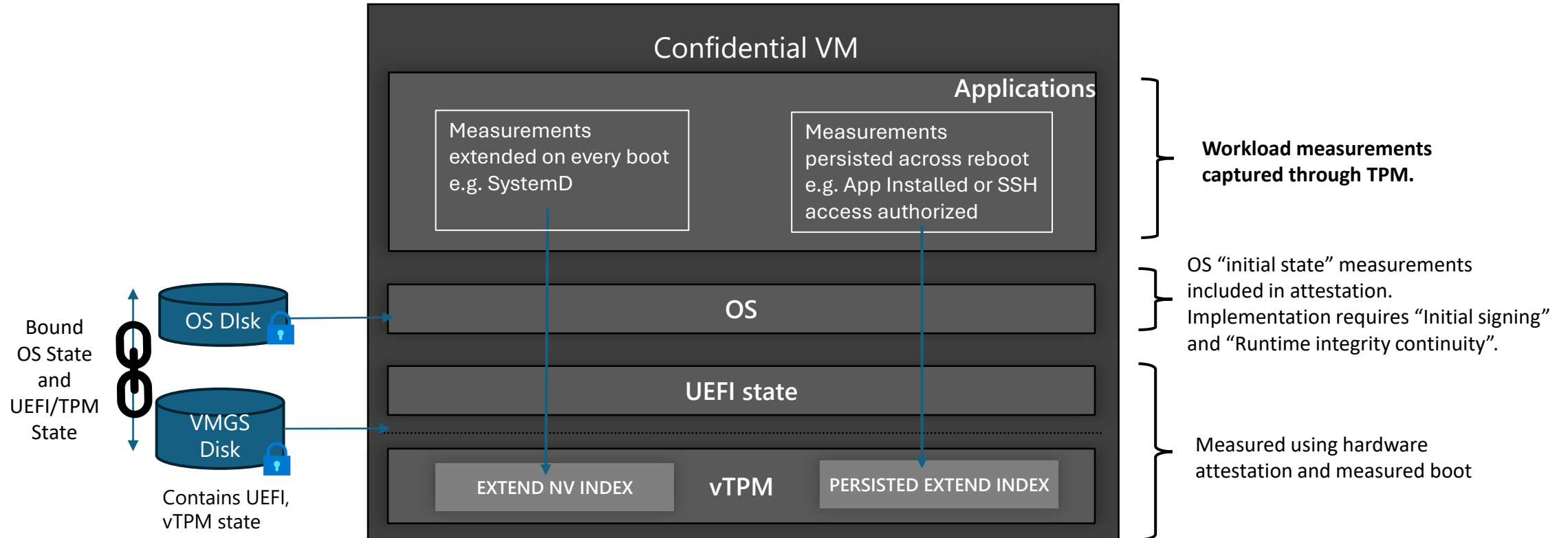
- To reason about security state of Virtual Machine's RW disks, we use RW Merkel.
 - Initial Signed Identity to establish trust in initial disk state at first boot/attach
 - Runtime integrity Continuity (using HMAC) across boot/attach → reboot/reattach.
- RW Merkel helps establish a Cryptographic Identity for RW disks.
 - TEEs can set boot or mount policies bound to the cryptographic identities.
- For OS disks, we rely on trusted boot environment to verify Initial Identity and establish Runtime Integrity and its continuity.
- The primitives also provide state binding of OS and UEFI/vTPM state.
- Call to Action
 - We are working with OS and Distro vendors, would love to work with other interested parties for joint exploration on design, implementation & standardization.
 - Looking for collaborators to explore Linux support for Production Ready RW Merkel.

Discussion

Clarifications and questions

Confidential VM – Using TPM

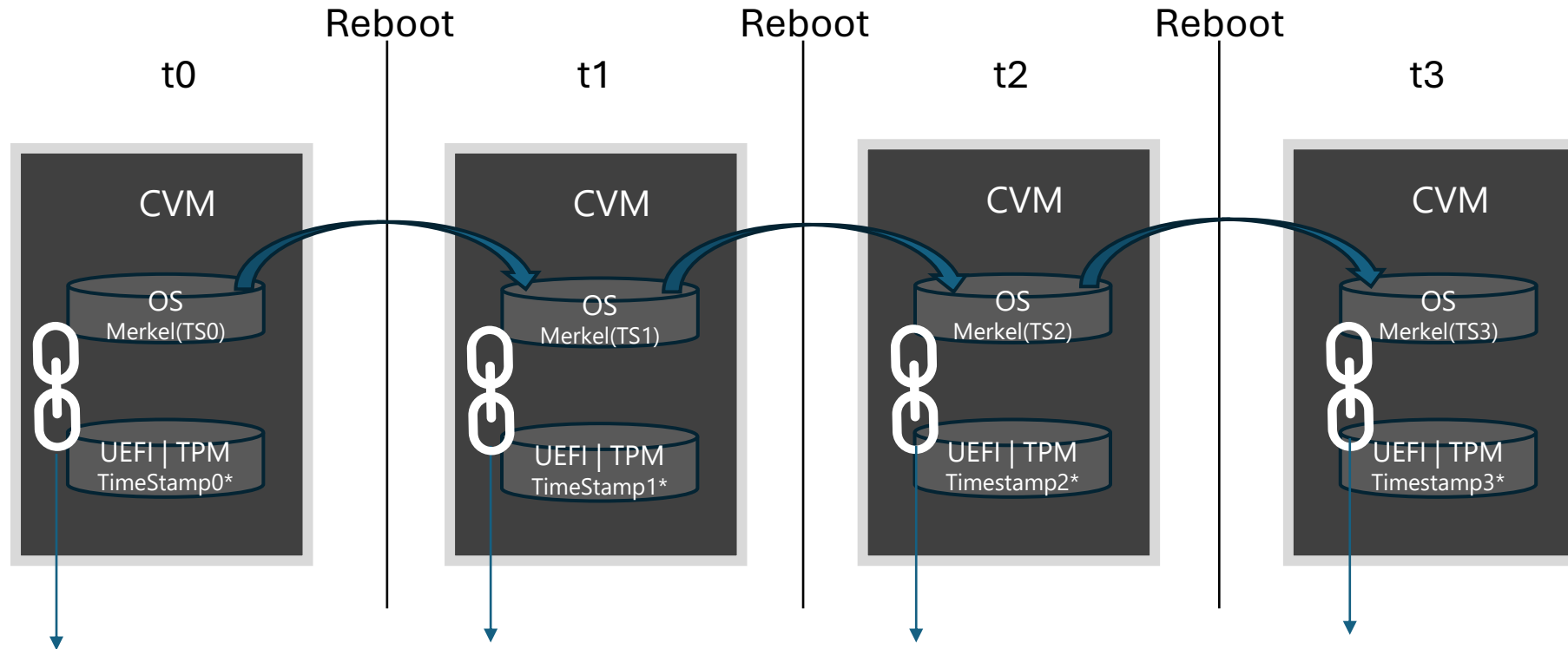
In persisted configuration, TPM can be used to extend and persist measurements across reboots.



- Extend NV Indexes used as “Faux PCRs”.
- Persisted NV Index persist state across reboots
- Prevent deletion through `TPMA_NV_POLICY_DELETE`.

OS rollback protection using Timestamp

OS and firmware state is cryptographically bound such that boot components can detect rollback of individual state



If RW Merkel is supported, runtime rollback protection is feasible using a “logical clock” mixed into Merkel root and written to OS disk as well as (periodically) written to UEFI state.

This guarantees rollback protection within specific “time” thresholds.